

MIT 805 Group Project: Part 2

Distributed Analysis of NYC Yellow Taxi Trips with PySpark



Mokgopo Lekatjile (u14098718)

Dlamini Msimisi (u18276319)

1 Processing Objective and Analytical Question

Taxi fleets earn money only while a vehicle carries a paying passenger, so where and when a driver spends their time determines their income. Part 1 profiled the data and showed that fares vary strongly with trip distance and season. Part 2 uses distributed processing to answer a decision-oriented question.

Central question: Which trip segments, pickup zones and hours of the day generate the most fare revenue and the best earnings per minute, and how should driver supply be allocated to match them?

The objectives are to (1) quantify how trips and fare revenue split across short, medium and long trips; (2) identify the pickup zones and hours with the highest demand and the highest earning efficiency (fare per minute and per mile); (3) measure seasonal and year-on-year change between 2024 and 2025; and (4) turn the findings into fleet allocation and policy recommendations. The question suits Spark because every answer requires grouping tens of millions of trips by a key (segment, zone, hour or month), which forces a distributed shuffle.

2 Mapping / Transformation Stage

The map stage turns each raw trip into a record carrying the key value pairs we need. All of these are *narrow* transformations: each output row depends on one input row, so Spark applies them partition by partition with no data movement.

```
# MAP / TRANSFORMATION (narrow transformations, no shuffle)
mapped_df = (
    processing_df
        .withColumn("trip_category",
            when(col("trip_distance") < 2, "Short")
            .when(col("trip_distance") < 10, "Medium")
            .otherwise("Long"))
        .withColumn("hour", F.hour("tpep_pickup_datetime"))
        .withColumn("is_weekend", F.dayofweek("tpep_pickup_datetime").isin(1, 7))
        .withColumn("minutes",
            (F.unix_timestamp("tpep_dropoff_datetime") -
             F.unix_timestamp("tpep_pickup_datetime")) / 60)
        .filter((col("minutes") > 1) & (col("minutes") < 360))
        .persist()
)
```

Three derived keys are created: trip_category (distance band), hour (hour of day) and is_weekend. The column minutes allows an earnings-per-minute measure, a better proxy for driver productivity than average fare, because a large fare earned over a long time may not be better than several short fares. The result is cached with persist() so the expensive deduplication shuffle runs only once. To show the classic MapReduce paradigm, the same logic was also written with the RDD API on a 10% sample (Python RDDs serialise every row, so they are impractical on all 83.6 million rows):

```
pairs = (
    mapped_df.sample(False, 0.1, 42)
        .select("PULocationID", "hour", "fare_amount", "trip_distance", "minutes").rdd
        .map(lambda r: ((r.PULocationID, r.hour),
            (1, r.fare_amount, r.trip_distance, r.minutes))) # MAP: emit key-value
```

The map emits a key-value pair ((zone, hour), (count, fare, miles, minutes)) per trip, which is the mapper output of Hadoop MapReduce.

3 Shuffle / Grouping Stage

To add up trips by zone, hour or month, Spark must first bring together trips that sit on different machines. This regrouping step is called the shuffle: each worker sorts its trips into "buckets" by label (zone, hour etc.), and the buckets are then sent to the workers who will total them. We grouped the data in four ways, so the shuffle happened four times:

Key	Purpose
trip_category	Compare segments (3 keys only, so very skewed: few groups, many rows each)
(PULocationID, hour)	Demand and earnings per zone per hour (about 260 zones x 24 hours = 6,191 groups)
(hour, is_weekend)	Daily demand profile, weekday versus weekend
(year, month)	Seasonal and year-on-year trend

Two design choices reduce the cost of the shuffle. First, Spark performs a partial aggregation (map-side) before the shuffle, so each partition sends one pre-summed row per key rather than millions of raw rows; this is the equivalent of a Hadoop combiner. Second, the zone lookup table (about 265 rows) is broadcast to every executor, so adding zone names needs no shuffle of the large table. We also lowered `spark.sql.shuffle.partitions` from the default 200 to 64, because the aggregated output is small and 200 tiny tasks add scheduling overhead. The three-valued category key causes data skew (a few partitions receive most rows), which is why the map-side combiner matters most for that query.

4 Reduction / Aggregation Stage

The reduce step collapses all records for a key into summary statistics. Two equivalent implementations were run. The RDD reduce adds the tuples; the function is associative and commutative, which lets Spark combine values locally before the shuffle:

```
agg_rdd = pairs.reduceByKey(lambda a, b: tuple(x + y for x, y in zip(a, b))) # SHUFFLE + REDUCE
```

The DataFrame aggregation runs on all 83.6 million rows and joins the zone names:

```
zone_hour = (
    mapped_df.groupBy("PULocationID", "hour")
    .agg(count("*").alias("trips"),
         spark_sum("fare_amount").alias("fare_revenue"),
         (spark_sum("fare_amount") / spark_sum("minutes")).alias("fare_per_min"),
         (spark_sum("fare_amount") / spark_sum("trip_distance")).alias("fare_per_mile"))
    .join(F.broadcast(zones), col("PULocationID") == zones.LocationID) # broadcast join: no shuffle of big side
```

Fare per minute is the ratio of sums, not the average of per-trip ratios; the ratio of sums weights each trip by its duration and is not distorted by very short trips with extreme per-minute values. Revenue means metered fare_amount only, excluding tips, tolls, surcharges and taxes. The same pattern produced the segment, hourly, monthly and payment summaries. Only these small aggregated results were converted to Pandas, for plotting.

5 Distributed Execution Evidence

The abridged physical plan for the zone-hour query (`explain("formatted")`) from the final run:

```
HashAggregate Keys [PULocationID, hour]
  Functions [partial_count(1), partial_sum(fare_amount), partial_sum(minutes),
partial_sum(trip_distance)]
Exchange hashpartitioning(PULocationID, hour, 64), ENSURE_REQUIREMENTS
HashAggregate Keys [PULocationID, hour]
  Functions [count(1), sum(fare_amount), sum(minutes), sum(trip_distance)]
BroadcastExchange HashedRelationBroadcastMode(...)
BroadcastHashJoin Left keys [PULocationID] Right keys [LocationID] Inner
AdaptiveSparkPlan isFinalPlan=true
```

From the scan upward, the first HashAggregate computes partial counts and sums inside each input partition (the map-side combine). The Exchange hashpartitioning(PULocationID, hour, 64) sends every record with the same (zone, hour) key to the same one of 64 shuffle partitions; this is the shuffle and the stage boundary. The second HashAggregate merges the partial results into final totals, which is the reduce. BroadcastExchange ships the 265-row zone table to every executor, so BroadcastHashJoin adds borough and zone names without shuffling the 83.6M-row side. Only pre-aggregated rows cross the network: 83.6 million trips were reduced to 6,191 zone-hour groups, and the job completed in 39.2 seconds on the cached dataset. The AdaptiveSparkPlan wrapper means adaptive query execution may revise the partition count at run time.

The Spark UI address printed in Colab is an internal container address that a browser cannot open, so the physical plan, partition settings, group count and job timing above are our execution evidence rather than UI screenshots. The notebook also runs the RDD version of the same aggregation on a 10 percent sample and prints its lineage with `toDebugString`.

Spark DAG versus Hadoop MapReduce.

Aspect	Hadoop MapReduce	Spark (this project)
Execution model	Fixed Map, Shuffle, Reduce per job	Directed acyclic graph of many transformations, split into stages at shuffle boundaries
Between steps	Map output written to local disk; each job result written to HDFS before the next job reads it	Narrow transformations are pipelined in memory within a stage; only shuffles write shuffle files
Multi-step analysis	One job per step, each re-reading the data	Several aggregations over one cached dataset (<code>persist()</code>), so the cleaning ran once

Aspect	Hadoop MapReduce	Spark (this project)
Evaluation	Job runs when submitted	Lazy: transformations build a plan; an action (show, count, toPandas) triggers execution, letting the Catalyst optimiser prune columns and reorder work
Combiner	Optional user-written class	Automatic partial aggregation

In our pipeline the scan, filters, derivations and partial aggregation form one pipelined stage, and the only stage boundary is the Exchange. In Hadoop each of those steps would be a separate job with intermediate results on disk. Spark still writes shuffle files, but it avoids the repeated reads and writes between chained jobs.

6 Results and Visualisations

6.1 Trip segments

Short trips make up 53.4% of trips but only 28.5% of fare revenue, while long trips are 8.1% of trips and 26.2% of revenue (Figure 1).

Segment	Trips	Share of trips	Avg fare	Fare revenue	Share of revenue	Fare per minute	Fare per mile
Short (< 2 mi)	44 642 463	53.4%	\$10.62	\$474.1M	28.5%	\$1.10	\$9.33
Medium (2 to 10 mi)	32 138 695	38.5%	\$23.42	\$752.7M	45.3%	\$1.10	\$5.72
Long (> 10 mi)	6 788 020	8.1%	\$64.20	\$435.8M	26.2%	\$1.37	\$4.06

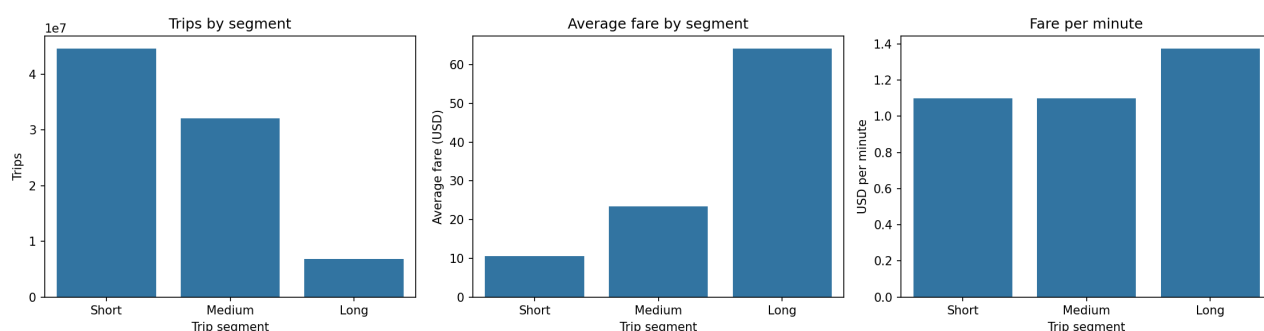


Figure 1. Trips, average fare and fare per minute by trip segment (2024 to 2025).

Medium trips are the largest revenue source (45.3%). Fare per minute is almost identical for short and medium trips (\$1.10 each) and about 25% higher for long trips (\$1.37), while fare per mile falls with distance (\$9.33, \$5.72, \$4.06) because the flag-drop and waiting charges are spread over more miles on longer trips. A driver who chains short trips therefore earns about the same per occupied minute as one doing medium trips; the difference is the time between fares, which the data does not record. The Pearson correlation between distance and fare on all 83.6M rows is only 0.14, far below the 0.85 we saw in an earlier 85 679 row sample. We suspect a small number of rows with implausible distance and fare combinations dominate Pearson on the full data, but we did not isolate them, so we do not draw conclusions from this coefficient and rely on the per-mile aggregates instead. It also shows that a sample correlation should not be assumed to hold on the full data.

6.2 Hour of day

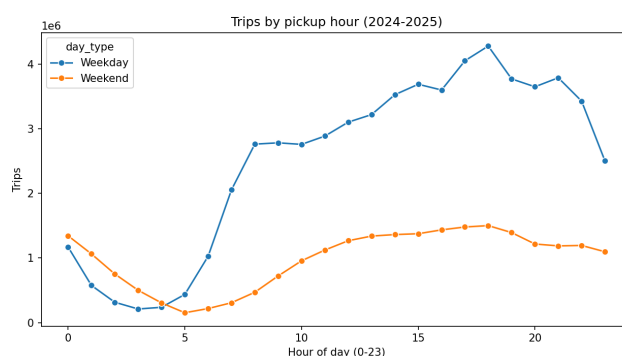


Figure 2. Trips by pickup hour, weekday versus weekend (2024 to 2025 totals).

Demand peaks at 18:00 on both weekdays (about 4.3 million trips over the two years) and weekends (about 1.5 million), with a second weekday peak at 17:00 (about 4.1 million) and a morning ramp-up from 1.0 million trips at 06:00 to 2.8 million at 08:00. Weekday volume is lowest at 03:00 to 04:00 (about 0.2 million per hour) and weekend volume at 05:00 (about 0.15 million). Between midnight and 04:00 weekends are busier than weekdays, for example 1.33 million versus 1.17 million at 00:00, consistent with nightlife demand. Weekday totals cover five days against two, so the per-day gap is much smaller than the figure suggests. The busiest hours are not the best paid: 14 of the 15 highest fare-per-minute zone-hour groups (with at least 5 000 trips) fall between 00:00 and 05:00, at \$1.96 to \$2.31 per minute against about \$1.10 for the average segment, and the other is at 23:00. Faster night-time travel is a plausible explanation, but we did not test it.

6.3 Zones

Demand is concentrated in a few Manhattan zones: 13 of the 15 busiest pickup zones are in Manhattan, led by Upper East Side South (3.86 million trips), Midtown Center (3.80 million) and Upper East Side North (3.43 million), with JFK Airport (3.71 million) third (Figure 3). Those Manhattan zones average only \$13.40 to \$19.58 per trip, so they supply volume rather than value per trip. The airports behave differently: JFK averages \$64.04 per trip and LaGuardia \$44.80, so JFK alone brings in \$237.3 million (14.3% of all fare revenue) from 4.4% of trips, and the two airports together bring in 20.9% of fare revenue from 7.4% of trips.

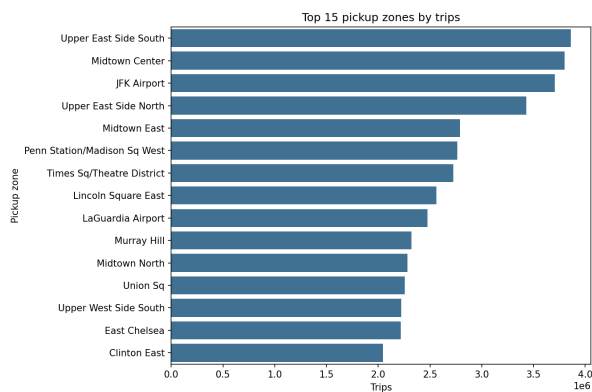


Figure 3. Top 15 pickup zones by trips.

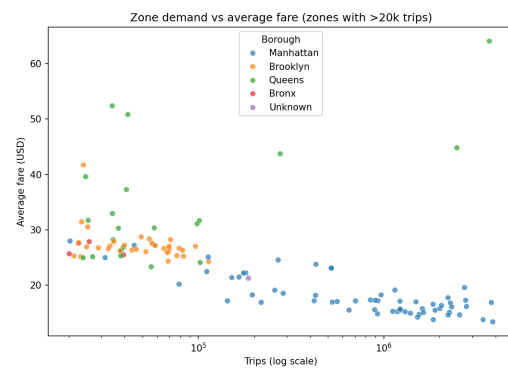


Figure 4. Zone demand versus average fare (zones with > 20,000 trips, log scale).

Figure 4 shows the trade-off: zones with more than 1 million trips are almost all in Manhattan at roughly \$13 to \$19 per trip, outer-borough zones in Queens and Brooklyn with 20,000 to 110,000 trips average roughly \$25 to \$52, and only the two airports combine high volume with a high fare. By borough (Figure 5), Manhattan dominates every hour and peaks at about 5 million trips at 17:00 to 18:00, Queens is a distant second at all hours because of the airports, and Brooklyn, the Bronx, Staten Island and Newark are negligible.

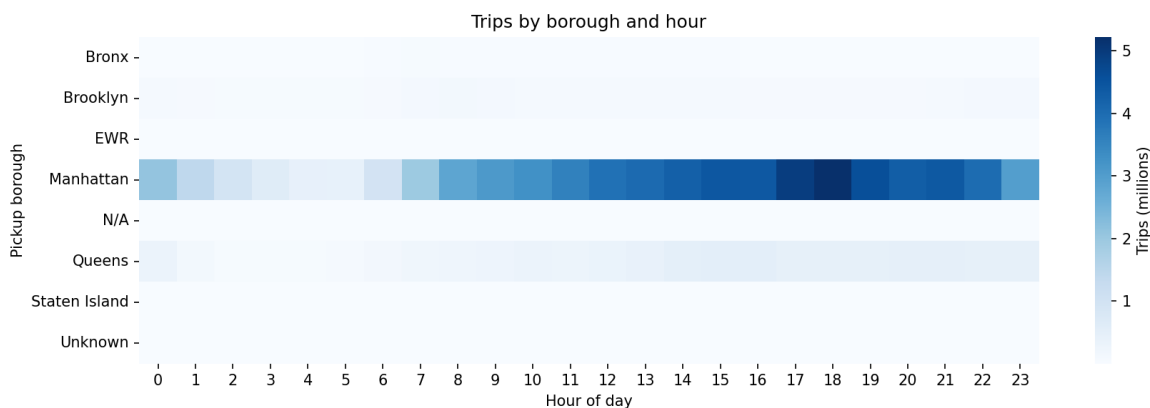


Figure 5. Trips by pickup borough and hour of day (millions).

6.4 Seasonality and growth

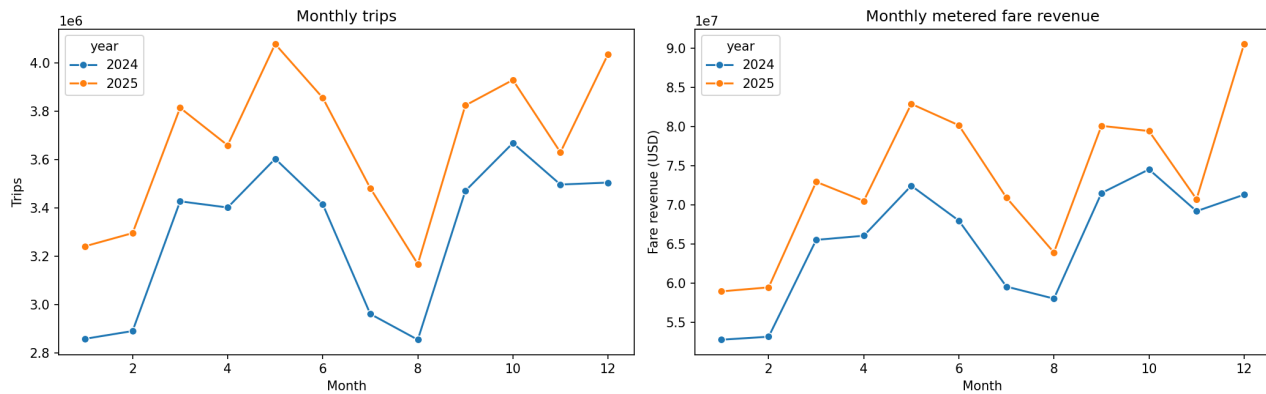


Figure 6. Monthly trips and metered fare revenue, 2024 versus 2025.

Monthly trips follow the same shape in both years: low in January and February, a spring peak in May, a trough in August and an autumn recovery (Figure 6). Trips grew 11.3% from 2024 to 2025 (39.56 million to 44.01 million) and fare revenue grew 12.6% (\$782.0 million to \$880.6 million); January to August alone grew 12.5%. Average fare rose within 2024 from \$18.45 in January to \$20.34 in December, and within 2025 from \$18.18 in January to \$20.95 in September. December 2025 is an outlier: 4.04 million trips, \$90.6 million in fare revenue (the highest month of both years) and an average fare of \$22.44, which is 15% above November 2025 (\$19.48) and 10% above December 2024 (\$20.34). We did not establish the cause, so we report it as unexplained and recommend a breakdown by segment as future work.

6.5 Payment type

Credit card accounts for 60.4 million trips (72.3%), Flex Fare 12.5 million (15.0%), cash 9.5 million (11.4%), disputed 0.88 million (1.1%) and no charge 0.30 million (0.4%); one trip has an unknown code. Average fares are similar for card (\$19.63) and cash (\$19.59), slightly higher for Flex Fare (\$21.03) and highest for disputed trips (\$24.26), which may mean higher-fare trips are more often contested. Code 0 is Flex Fare, not cash, and the Part 1 report mislabelled it. Because cash tips are not recorded, tips are understated for the 11.4% of trips paid in cash. We have no time series by payment type, so we make no claim about a trend in cash use.

	payment_type	trip_count	avg_fare	label	pct
0	1	60378973	19.634265	Credit card	72.250289
1	0	12522305	21.033352	Flex Fare	14.984358
2	2	9493592	19.589925	Cash	11.360160
3	4	879050	24.264399	Dispute	1.051883
4	3	295257	21.874893	No charge	0.353308
5	5	1	51.800000	Unknown	0.000001

7 Business and Societal Interpretation

Fleet operators and drivers

The results indicate where and when to position vehicles, and three allocation rules follow. First, keep Manhattan core supply (Upper East Side, Midtown, Penn Station) high from 07:00 to 10:00 and from 16:00 to 20:00, where weekday demand peaks; these zones supply volume rather than value per trip. Second, move part of the night fleet (00:00 to 05:00) to JFK, LaGuardia and Midtown, where fare per minute is about \$2.0 to \$2.3, roughly double the \$1.10 segment average. Third, treat airports as a deliberate revenue strategy: 7.4% of trips yield 20.9% of fare revenue, but airport queues and empty return legs are not recorded, so the true hourly gain is lower than fare per minute suggests. Because long trips pay about 25% more per minute, drivers who accept trips beyond 10 miles gain only if they can find a return fare.

City planners and regulators

Concentration of pickups in a small number of zones and hours shows where congestion pricing, curb management and taxi stands have the most effect. The 11 percent year-on-year growth in trips shows demand recovering and supports capacity planning.

Passengers

Better supply matching helps shorten wait times in under-served zones and reduces cruising, which lowers emissions and congestion; and potentially crime.

Limitations

1. Fare revenue excludes tips, tolls and surcharges, and cash tips are not recorded at all, so true driver earnings are understated, especially for cash and Flex Fare trips.
2. Yellow taxis only: App-based services such as Uber and Lyft, and green cabs, are excluded, so we see one slice of the market and any shift of demand between services is invisible.
3. Earnings per minute ignores idle time: The data records occupied trips only, so we cannot measure time spent searching for a passenger, which determines true hourly income.
4. Subset and sampling: Only 2024 and 2025 were processed and the RDD demonstration used a 10 percent sample. Two years cannot separate a trend from a once-off shock such as December 2025.
5. Data quality: Driver-reported and system-generated fields may be inaccurate, our distance and duration filters are judgement calls, and the weak full-data distance-fare correlation (0.14) shows residual implausible records remain.

The analysis is descriptive: high-earning zones and hours are associations, and we make no causal claims about why they differ.

8 Reproducibility

The repository (<https://github.com/michael-dlam/project>) follows the required structure (README.md, requirements.txt, data/README.md, notebooks/, src/, output/, figures/, report/). Data is not committed; data/README.md gives the TLC download URL pattern and the zone lookup link, and the notebook downloads all 24 Parquet files programmatically. Dependencies are pyspark, pandas, matplotlib, seaborn and requests. Running the notebook top to bottom reproduces every table and figure.

9 References

NYC Taxi and Limousine Commission. TLC Trip Record Data. <https://www.nyc.gov/site/tlc/about/tlc-trip-recorddata.page>

NYC TLC. Yellow Taxi Trip Records Data Dictionary.

https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf